

Role Description Cards

Five-Person Team

Working roles for coordination, not walls: every team member should be able to run the code and read the report.

These are **suggested** role descriptions, not a rigid requirement — they exist to help a five-person team self-organise. You may rename, merge, or redistribute the responsibilities to fit your team; what matters is that (i) every deliverable has an owner, and (ii) nobody waits for an upstream “expert” before starting. Roles rotate between projects so that everyone experiences a different slice of the work.

Each card below says what the role typically owns, what “done” looks like, and how it is checked. Print these on cards or paste them on the module homepage at the start of Week 1.

① Project Manager

Owns: the plan, the meetings, the presentation, the git setup, and the weekly records (challenge consensus + milestone).

- Runs a 15-minute team meeting before each Tutorial (arrive a little early) or at the start of the tutorial’s team-work block — the opening micro-lecture slot is not available for it
- Maintains a simple Gantt table (what / who / due) on the module homepage
- Makes sure every teammate has a named task for the week
- Creates the git repository and the folder structure at the start of Week 1 (see 04_Tutorials/git_quickref), so all collaboration happens through git from day one
- Invites every teammate *and the teacher* as repository collaborators (Write access) in Week 1, so everyone — teacher included — can clone and push from day one
- Leads the Seminar presentations (or coordinates the speakers)
- Records the team’s **challenge consensus** at each Tutorial’s review — the one result everyone agrees on
- Posts the team’s **Weekly-Milestone** row at each wrap-up (what we completed / what we plan next / current blocker)
- Posts the **team result on the course forum** after each Tutorial’s review (one post per team per week — the team record)
- Coordinates the final submission (demo + repo + report deadline)

“Done” looks like: the Gantt table is updated each Tutorial; the git repository exists with a sensible code/, figures/, report/ layout and every teammate has pushed at least once by the end of Week 1; the presentation slides exist before the assigned seminar; the Weekly-Milestone row and the team’s forum post appear every week; the final submission is in on time; nobody in the team is idle in a tutorial.

Checked by: the team’s checkpoints each week + presentation quality + the weekly milestone and forum posts + the challenge-consensus rows.

② Mathematical Theory

Owens: every formula that appears in the report.

- Derives (or finds and checks) the ODE model $f(t, y)$
- Derives the absolute-stability regions and the local truncation errors
- Computes the linearised Jacobian / stiffness ratio for the report
- Proofreads all equations in the final $\LaTeX$

“Done” looks like: the report’s *Model* and *Stability* sections contain derivations that reproduce known results (e.g. Euler stability region = disk centered at -1).

Checked by: seminar Q&A — be ready to derive on the whiteboard the stability condition you claim.

③ Algorithm Implementation

Owens: the code that solves the problem.

- Implements Euler, RK4, implicit Euler (with Newton), and the adaptive controller
- Structures the code so $f(t, y)$ can be swapped for any topic
- Commits regularly with meaningful messages (commit at the end of each session, in the PM’s repository layout)
- Keeps `run_all.py` running end-to-end to reproduce all figures

“Done” looks like: `python code/run_all.py` runs on a machine with dependencies installed and regenerates every figure in the report.

Checked by: code-review checklist (Tutorial 3) + the viva questions.

④ Visualization & Report

Owens: every figure and the written report.

- Turns raw outputs into publication-quality figures (log-log, labelled, no rainbow colormaps — see the visualization guide)
- Produces the failure-case animation if your topic calls for one
- Typesets the $\LaTeX$ report and the slides
- Keeps the English clean and the narrative clear

“Done” looks like: the report reads like a detective story (experiments $\rightarrow$ evidence $\rightarrow$ interpretation), and every figure has a caption explaining what is being shown and why it matters.

Checked by: the visualization checklist + reviewer comments.

⑤ Testing & Validation

Owens: whether the results can be *believed*.

- Verifies the code against a reference solution the team constructs itself (see the brief's validation section)
- Compares every claimed number against the team's own reference solution
- Hunts edge cases: step size too large, Newton from a bad guess, adaptivity tolerance too loose, negative concentrations, non-physical values
- Runs the code-review checklist against the team's own code before Tutorial 3

“Done” looks like: `python code/run_all.py` regenerates every figure from a machine with dependencies installed, and the report's key claims are each backed by a comparison to the team's own reference solution or a figure.

Checked by: the instructor runs the code; the numbers in the report should match the team's own reference solutions.

Rotation note

We suggest rotating roles between projects so that every student gets to try three different roles by the end of the course. If someone is clearly the strongest coder, it is still worth letting someone else take the Algorithm Implementation seat in a later project — but follow your team's judgement.

What all five share

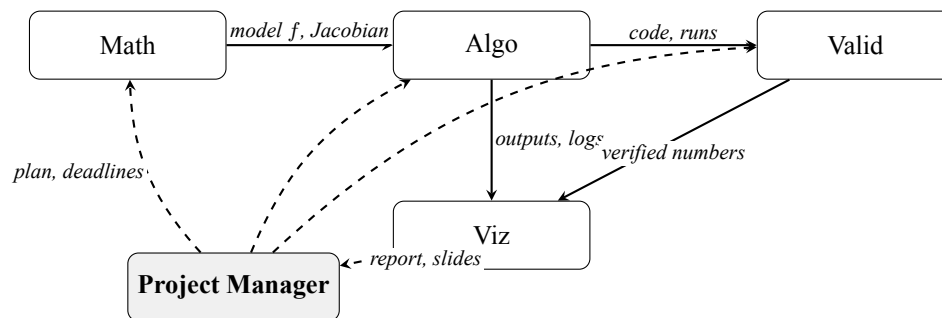
- You should be able to explain the *core* algorithm even if it is not “your” part
- You attend every session and submit your issue ticket before each Tutorial
- You write your Individual Contribution Statement (ICS) for the final report
- You are honest about what you did and did not do

Example work plan — parallel, not serial

All five roles start in Week 1; each one's deliverables mature at different weeks. The columns below are *not* a hand-off sequence — they show what each person is expected to have moving in a given week.

	Week 1	Week 2	Week 3	Week 4
PM	git repo + folder structure, roles + Gantt	checkpoints, de-risk + plan	rehearse the seminar talk	demo coordination, submission
Math	fix model $f(t, y)$, Jacobian	stability regions, LTE derivation	closed-form / analytic cases	proofread, viva prep
Algo	Euler skeleton + git, run on trivial f	RK4 + implicit Euler + Newton	adaptive controller, refactor	run_all.py end-to-end
Valid	build oracle (scipy), test plan	unit tests vs. oracle	edge cases, convergence order	final check, code review
Viz	report template, figure skeleton	first plots (convergence)	main figures + slides	final report, animation

How the five roles interact



Reading the diagram: solid arrows are the data flow (the model defines the code, the code produces runs, validation checks them, plots and the report consume everything). Dashed arrows are the Project Manager's coordination loops — planning, deadlines, and the presentation. Notice that *validation starts in Week 1* (building the oracle) and *plots start in Week 2*: nobody waits for a finished upstream.